POLITECNICO
MILANO 1863

DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

Internet Of Things

Challenge 2 – Packet Sniffing

* Seyedaman Zargari – 11109586
Amirhossein Dibaj – 11109540

Academic Year 2025–2026

Contents

1 CQ1 – NON DELETE Requests to coap.me **3**
1.1 CQ1a 3
1.2 CQ1b 3

2 CQ2 – Resources with Equal Unsuccessful CON POST and PUT **3**

3 CQ3 – Observe Notifications for /dining_room/temperature **3**
3.1 CQ3a 3
3.2 CQ3b 4

4 CQ4 – MQTT-SN Messages from Local Broker (Port 1885) **5**

5 CQ5 – Last Will via Wildcard Subscription **5**

6 CQ6 – Erasing Retained Values on HiveMQ **5**
6.1 CQ6a 5
6.2 CQ6b 5

7 CQ7 – Subscribe Requests with Multiple Wildcards **6**

8 CQ8 – Topic Depth Histogram (Both Files) **6**

1 CQ1 – NON DELETE Requests to coap.me

1.1 CQ1a

Filter: All NON-Confirmable (type=1) DELETE (code=4) requests directed to `coap.me` (IP `134.102.218.18`, resolved via DNS in the capture), matched with their responses by token.

Out of 20 NON DELETE requests to `coap.me`, only **1** received a successful (2.xx) response:

MID	Resource	Response Code	Meaning
30800	/validate	2.02	Deleted

Table 1: NON DELETE requests with successful response

The remaining 19 requests all received `4.05 Method Not Allowed`.

Answer CQ1a: 1 request. **MID = 30800**.

1.2 CQ1b

Although the server responded with `2.02 Deleted` to MID 30800, a subsequent GET request to `/validate` (frame 11363) still received a `2.05 Content` response (frame 11364), proving the resource was not actually deleted. `coap.me` is a public test server that acknowledges the DELETE method on `/validate` but does not truly remove the resource.

Answer CQ1b: 0

2 CQ2 – Resources with Equal Unsuccessful CON POST and PUT

Method: Extracted all Confirmable (type=0) POST (code=2) and PUT (code=3) requests directed to the local server (`127.0.0.1:5683`). Matched each with its response (by token, from source port 5683). Counted requests as unsuccessful only when a 4.xx or 5.xx response was present. Used MID as uniqueness criterion (retransmissions share the same MID).

Only `/dining_room` has $X = Y = 1$ (POST got 4.00 Bad Request; PUT got 4.04 Not Found).

Answer CQ2: 1

3 CQ3 – Observe Notifications for /dining_room/temperature

3.1 CQ3a

Method: Identified the observe registration (frame 4343: CON GET with `Observe=0`, token `89b9`, to `127.0.0.1:5683`). Then collected all responses from the server (srcport 5683) matching this token with the Observe option present. Grouped by MID to eliminate retransmissions.

Resource	Unique Unsucc. POST (X)	Unique Unsucc. PUT (Y)	X=Y?
/basic	2	1	No
/dining_room	1	1	**Yes**
/dining_room/door	2	3	No
/dining_room/temperature	5	2	No
/hello_post	3	0	No
/hello_world	4	0	No
/living_room	1	0	No
/living_room/door	0	1	No
/living_room/temperature	1	0	No
/main_door	1	2	No
/test	2	0	No

Table 2: Unique unsuccessful CON POST/PUT per resource on local server

#	MID	Type	Observe	Temp (°C)	ACKed	Retransmissions
1	10588	ACK	1	19.97	Yes	0
2	35	CON	1	22.32	Yes	0
3	38	CON	2	18.85	Yes	0
4	43	CON	3	25.98	Yes	0
5	45	CON	4	15.59	Yes	0
6	49	CON	5	25.35	Yes	0
7	54	CON	6	26.21	**No**	7
8	62	CON	7	27.53	**No**	3
9	74	CON	8	26.98	**No**	1
10	87	CON	9	25.32	**No**	1
11	90	CON	10	19.57	**No**	1

Table 3: Unique observe notifications for `/dining_room/temperature`

Answer CQ3a: 11

3.2 CQ3b

Starting from notification #7 (MID 54, Observe=6), the client stops acknowledging CON notifications. MID 54 is retransmitted 7 times without receiving an ACK. According to RFC 7641 §4.5, when a CON notification fails (exhausts retransmissions without ACK), the server should cancel the observe relationship. Despite this, the server continues sending notifications #8–11 (MIDs 62, 74, 87, 90), none of which are ever acknowledged. All 5 unacknowledged notifications (MIDs 54, 62, 74, 87, 90) represent wasted traffic.

Answer CQ3b: 5

4 CQ4 – MQTT-SN Messages from Local Broker (Port 1885)

Filter: Configured tshark to decode UDP port 1885 as MQTT-SN (`-d udp.port==1885,mqttsn`). Filtered for packets with `udp.srcport == 1885` (messages FROM the broker to clients).

Analysis: 219 MQTT-SN messages were sent by clients *to* port 1885 (`udp.dstport == 1885`). Every single one triggered an ICMP "Destination Unreachable – Port Unreachable" (Type 3, Code 3) response, confirming that no broker was listening on port 1885. Zero packets were observed with `udp.srcport == 1885`, meaning the broker never sent any response.

Answer CQ4: 0

5 CQ5 – Last Will via Wildcard Subscription

Method: Identified 4 CONNECT packets with Last Will flags set. Verified that all 4 clients disconnected ungracefully (no DISCONNECT message found in their TCP streams), meaning all 4 last wills were triggered. Then matched each will topic against all wildcard SUBSCRIBE topics on the same broker.

Will Topic	Broker	Matching Wildcard Sub	Subscriber Strea
university/department12/room1/temperature	local (::1)	`university/#`	10
metaverse/room2/floor4	mosquitto	(none)	–
hospital/facility3/area3	mosquitto	(none)	–
metaverse/room2/room2	mosquitto	(none)	–

Table 4: Last will topics and wildcard subscription matches

Only the local broker will topic is matched by a wildcard subscription (`university/#` on stream 10).

Answer CQ5: 1

6 CQ6 – Erasing Retained Values on HiveMQ

6.1 CQ6a

Method: Filtered for PUBLISH messages to HiveMQ (IPs `18.192.151.104`, `35.158.43.69`) with `retain=1` and empty payload (`payload_length = 0`). An empty retained publish erases the previously stored value for that topic.

Three distinct TCP streams (clients) sent such messages: streams 1, 22, and 43.

Answer CQ6a: 3

6.2 CQ6b

Method: Extracted the Client ID from the CONNECT packet of each stream identified in CQ6a.

Stream	Client ID	Length (bytes)	> 7?
1	pcbtjyj	7	No
22	giuxfijwus	10	**Yes**
43	atqeblw	7	No

Table 5: Client IDs of clients erasing retained values

Answer CQ6b: 1

7 CQ7 – Subscribe Requests with Multiple Wildcards

Filter: SUBSCRIBE packets (`mqtt.msgtype == 8`) directed to the local broker (`ipv6.dst == ::1`), where at least one topic filter contains two or more wildcard characters (`+` or `#`).

Nine such SUBSCRIBE requests were found:

Frame	Topic Filter
226	university/+/+/deposit
363	hospital/+/+/light
460	metaverse/+/room1/#
473	house/department3/+/+
700	metaverse/facility0/+/+
1177	hospital/fxlugs/+/#
1915	metaverse/+/+/hydraulic_valve
2360	factory/+/#
2994	house/+/room1/#

Table 6: Subscribe requests with ≥ 2 wildcards to local broker

Answer CQ7: 9

8 CQ8 – Topic Depth Histogram (Both Files)

Method: Extracted all MQTT PUBLISH messages (`mqtt.msgtype == 3`) directed to the local broker in each capture:

- **A.pcapng:** local broker at `127.0.0.1:1883`
- **B.pcapng:** local broker at `[::1]:1883`

For each message, the number of topic layers was computed by counting segments separated by `/` (e.g., `university/room2/floor4` → 3 layers).

Number of Layers	A.pcapng	B.pcapng
1	0	21
2	154	172
3	131	162
4	154	179

Table 7: Topic layer distribution for PUBLISH messages to local broker

CQ8a (total publishes in A.pcapng): **439**
CQ8b (total publishes in B.pcapng): **534**

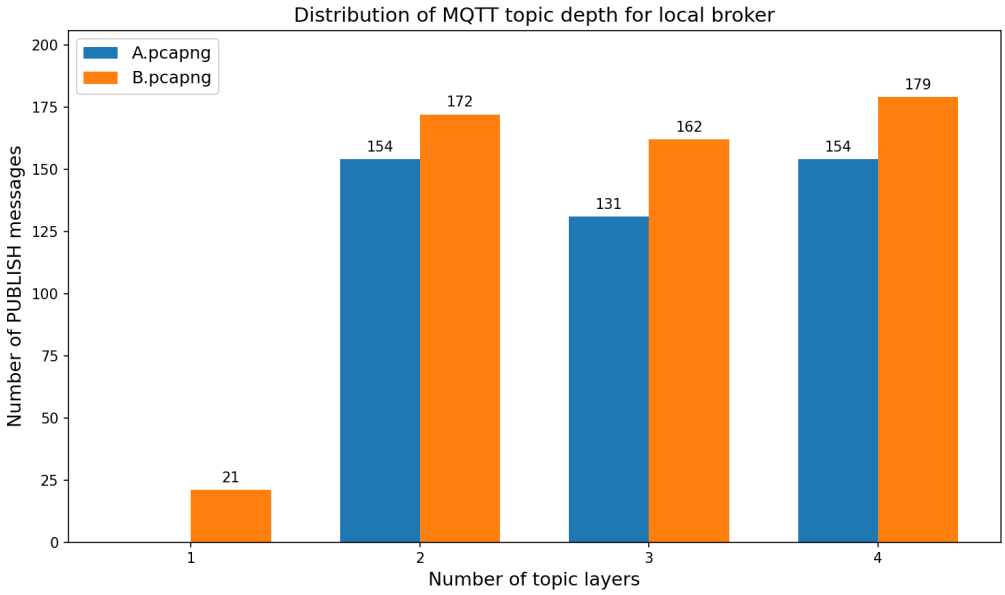

Figure 1: Distribution of MQTT topic depth for local broker

B.pcapng shows a broader distribution with topics at all depths 1–4, including 21 single-layer topics not present in A.pcapng. Both captures concentrate most traffic at 2–4 layers, with B.pcapng having a consistently higher message count across all depths.